

# SOFTWARE ARCHITECTURE AND DESIGN SPECIFICATION

Group Name: **Voice Assistant for Multimedia Controls**

Version: 1.0

Date: 03 October 2026

## Revision History

| Version | Date            | Author   | Change Summary                                 |
|---------|-----------------|----------|------------------------------------------------|
| 1.0     | 03 October 2026 | Group 12 | Initial architecture and design specification. |

## Approvals

| Sl. No. | Name of Team Member | SRN           | Role        |
|---------|---------------------|---------------|-------------|
| 1       | Prajwal GT          | PES2UG25CS818 | Team Member |
| 2       | Shreedhar R         | PES2UG25CS823 | Team Member |
| 3       | Shrinivas S J       | PES2UG24CS494 | Team Member |
| 4       | Sarang S Nair       | PES2UG24CS447 | Team Leader |

## 1. Introduction

### 1.1 Purpose

This document specifies the software architecture and design of the Voice Assistant for Multimedia Controls. It defines major modules, responsibilities, interfaces, architecture decisions, technology stack, risks, traceability, security and detailed design.

### 1.2 Scope

The system provides hands-free multimedia control using predefined voice commands. Core functions are play/resume, pause, volume up, volume down, next and previous. Optional MediaPipe gesture recognition provides an additional

control mechanism. The system is intended for a Raspberry Pi or ESP-based controller connected to a computer or multimedia device.

### 1.3 Audience

- Developers implementing the system.
- QA/test engineers.
- Faculty/evaluators.
- Project maintainers.
- Group 12 members responsible for integration and demonstration.

### 1.4 Definitions

| Term               | Definition                                      |
|--------------------|-------------------------------------------------|
| Voice Command      | Predefined spoken instruction.                  |
| Media Control      | Play, pause, resume, volume or track operation. |
| Raspberry Pi / ESP | Embedded controller platform.                   |
| MediaPipe          | Optional hand-gesture recognition framework.    |
| SRS                | Software Requirements Specification.            |
| STP                | Software Test Plan.                             |
| RTM                | Requirements Traceability Matrix.               |
| API                | Application Programming Interface.              |

## 2. Document Overview

### 2.1 How to use this document

Sections 1–2 introduce the project. Section 3 describes architecture, stakeholders, components, architecture pattern, technology, risks, traceability and security. Section 4 describes detailed design and interfaces. Section 5 contains appendices.

### 2.2 Related Documents

| Document         | Purpose                                            |
|------------------|----------------------------------------------------|
| SRS              | Functional and nonfunctional requirements.         |
| STP              | Testing strategy, test cases and responsibilities. |
| Project Proposal | Project scope, work plan and product ownership.    |
| RTM              | Requirement-to-test traceability.                  |

## 3. Architecture

### 3.1 Goals & Constraints

Goals:

- Hands-free multimedia control.
- Responsive command execution.
- Modular and maintainable design.
- Optional gesture control.
- Safe handling of invalid input.

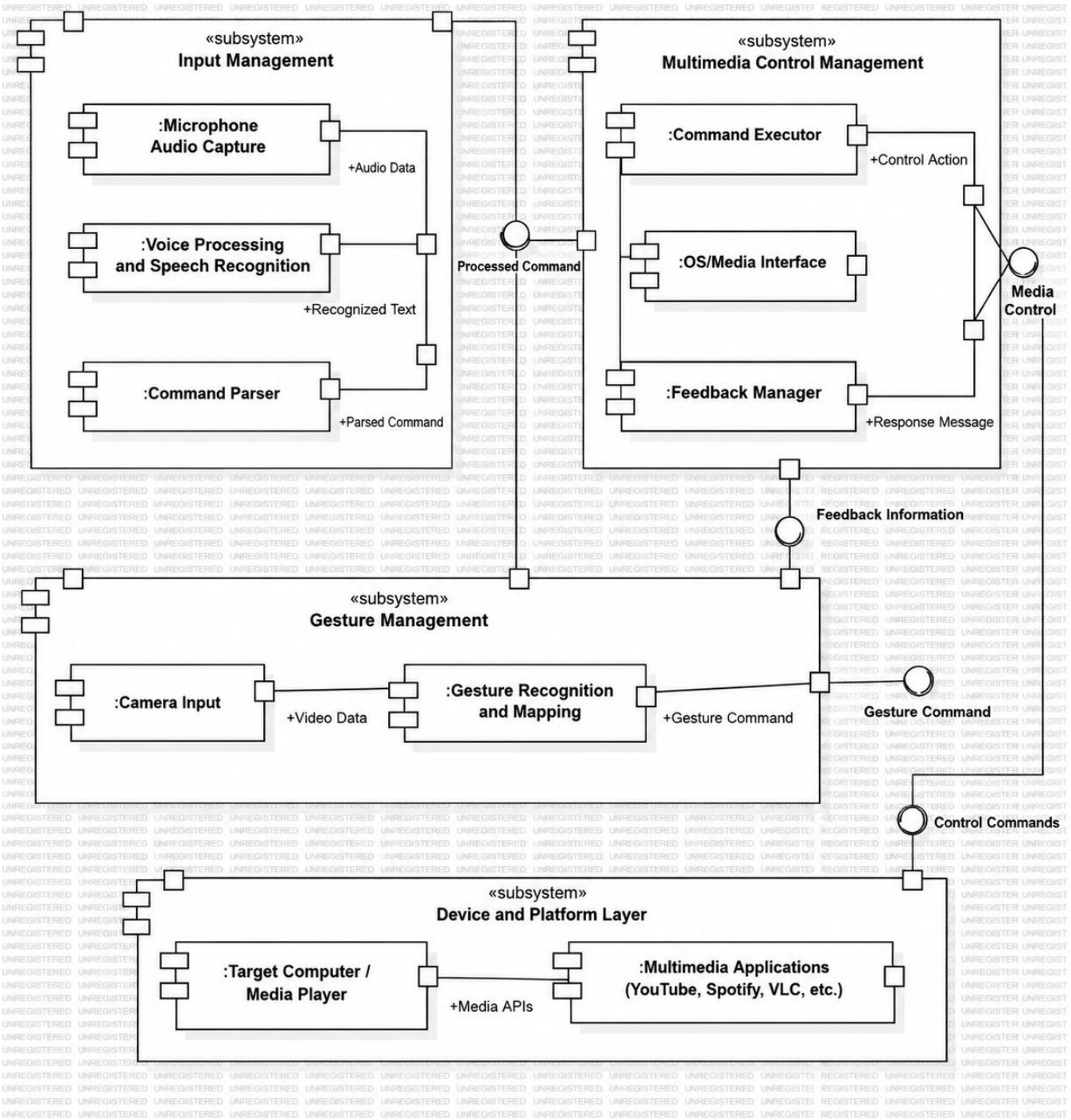
Constraints:

- Limited Raspberry Pi/ESP resources.
- Voice recognition affected by noise and microphone quality.
- Target device must support the selected control interface.
- MediaPipe requires compatible camera/software.
- Initial release uses predefined commands.

### 3.2 Stakeholders & Concerns

| Stakeholder          | Concerns                                  |
|----------------------|-------------------------------------------|
| Users                | Usability, responsiveness, reliability.   |
| Project Team         | Modularity, integration, testability.     |
| Faculty / Evaluators | Correct design and working demonstration. |
| Maintainers          | Clear interfaces and extensibility.       |
| Target Device        | Correct multimedia actions.               |

3.3 Component (UML) Diagram



3.4 Component Descriptions

| Component                  | Responsibility                               |
|----------------------------|----------------------------------------------|
| Voice Input Module         | Captures microphone input.                   |
| Command Recognition Module | Recognizes supported commands.               |
| Command Mapping Module     | Maps commands to actions.                    |
| Media Control Module       | Executes play, pause, volume and navigation. |
| Feedback Module            | Provides command/action status.              |

|                                   |                                                 |
|-----------------------------------|-------------------------------------------------|
| Gesture Control Module (Optional) | Recognizes configured gestures using MediaPipe. |
| Device Communication Layer        | Sends actions to the target device.             |

### 3.5 Chosen Architecture Pattern and Rationale

A modular layered architecture is selected. Input, recognition, command mapping, multimedia control and device communication are separated. This improves maintainability, testing and future replacement of individual components.

### 3.6 Technology Stack & Data Stores

| Area            | Choice                                             |
|-----------------|----------------------------------------------------|
| Controller      | Raspberry Pi or ESP                                |
| Language        | Python 3.x                                         |
| Voice           | Python-compatible voice-recognition library        |
| Gesture         | MediaPipe, optional                                |
| Inputs          | Microphone; optional camera                        |
| Target          | Computer / multimedia device                       |
| Version Control | Git                                                |
| Data Store      | No persistent database required for core prototype |

### 3.7 Risks & Mitigations

| Risk                         | Mitigation                                                   |
|------------------------------|--------------------------------------------------------------|
| Noise affects recognition    | Use clear commands and test normal noise.                    |
| Microphone failure           | Verify input and keep tested replacement if available.       |
| Device communication failure | Perform early integration testing and handle failure safely. |
| Media-player differences     | Select and document a supported target.                      |
| MediaPipe compatibility      | Use compatible environment and isolate gesture module.       |
| Unintended action            | Ignore unsupported/uncertain input.                          |

### 3.8 Traceability to Requirements

| Requirement  | Component                         |
|--------------|-----------------------------------|
| REQ-VC-01/02 | Voice Input + Command Recognition |
| REQ-VC-03/04 | Media Control                     |
| REQ-VC-05/06 | Media Control                     |
| REQ-VC-07/08 | Media Control                     |
| REQ-VC-09    | Feedback                          |
| REQ-VC-10    | Recognition + Mapping             |
| REQ-G-01/02  | Gesture Control                   |
| REQ-G-03     | Gesture Mapping                   |
| REQ-G-04     | Gesture Control                   |

### 3.9 Security Architecture

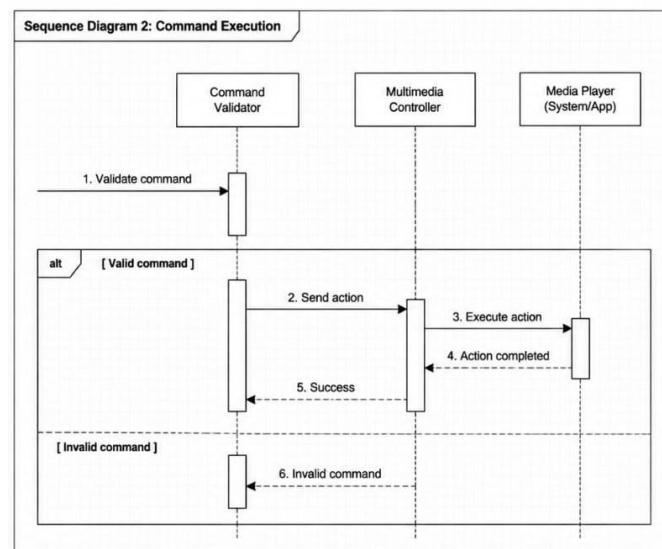
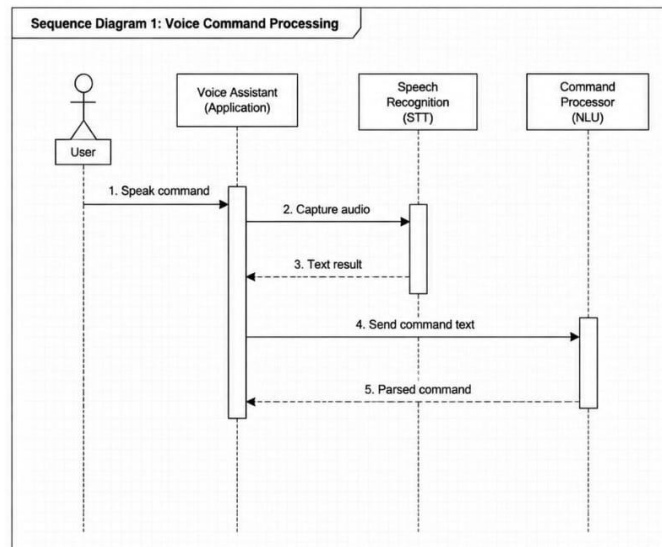
Only configured commands and gestures should trigger actions. Unsupported or uncertain input is rejected. Voice recordings should not be unnecessarily stored. If an external voice service is used, its privacy requirements must be considered. Configuration should not be accidentally modified during normal operation.

## 4. Design

### 4.1 Design Overview

Input modules capture voice or optional gesture data. Recognition converts input into a command or gesture identifier. Mapping selects the multimedia action. The media-control module executes it through the target-device interface. Feedback reports status.

### 4.2 UML Sequence Diagrams



#### 4.2.1 Voice Command Flow

- User speaks a supported command.
- Microphone captures audio.
- Voice Input sends input for recognition.
- Command Recognition identifies the command.
- Command Mapping selects the action.
- Media Control sends the action to the target device.
- Target device performs the action.
- Feedback reports status where enabled.

#### 4.2.2 Gesture Control Flow (Optional)

- User performs a configured gesture.
- Camera captures a frame.
- MediaPipe processes the frame.
- Gesture is identified.
- Gesture Mapping selects an action.
- Media Control sends the action.
- Target device performs it.
- Unsupported/uncertain gestures are ignored.

### 4.3 API / Interface Design

| Interface          | Operation   | Input        | Output                |
|--------------------|-------------|--------------|-----------------------|
| VoiceInput         | listen()    | Microphone   | Audio/text            |
| CommandRecognition | recognize() | Audio/text   | Command ID            |
| CommandMapping     | map()       | Command ID   | Action ID             |
| MediaControl       | execute()   | Action ID    | Status                |
| Feedback           | notify()    | Status       | User feedback         |
| GestureControl     | detect()    | Camera frame | Gesture ID            |
| DeviceControl      | send()      | Action       | Acknowledgement/error |

### 4.4 Error Handling, Logging & Monitoring

- Unsupported voice input produces no action.
- Microphone failure is reported without crashing.
- Communication failures return an error and avoid unintended repeated actions.
- Unsupported gestures are ignored.
- Logs contain diagnostic status/errors without unnecessary sensitive voice content.
- Monitor recognition failures, device-control failures and repeated errors.

### 4.5 UX Design

The interface is primarily voice-based. Commands should be short and consistent. Optional audio or visual feedback confirms recognition. Gesture mode should have a clear active indication. Normal operation should not require complex menus.

### 4.6 Open Issues & Next Steps

- Finalize exact Raspberry Pi/ESP hardware and target media-player interface.
- Select and validate the voice-recognition library.
- Implement and test core media control.
- Finalize MediaPipe gestures.

- Complete integration and system testing.
- Document final hardware and configuration.

## 5. Appendices

### 5.1 Glossary

| Term          | Meaning                                   |
|---------------|-------------------------------------------|
| Voice Command | Predefined spoken instruction.            |
| Media Control | Playback, volume or navigation operation. |
| MediaPipe     | Optional gesture framework.               |
| SRS           | Software Requirements Specification.      |
| STP           | Software Test Plan.                       |
| RTM           | Requirements Traceability Matrix.         |
| API           | Application Programming Interface.        |

### 5.2 References

- Group 12 SRS – Voice Assistant for Multimedia Controls.
- Group 12 STP – Voice Assistant for Multimedia Controls.
- Group 12 Project Proposal – Voice Assistant for Multimedia Controls.
- Provided Software Architecture and Design Specification template.

### 5.3 Tools

| Tool               | Use                                     |
|--------------------|-----------------------------------------|
| Python             | Application development                 |
| Git                | Version control                         |
| VS Code / IDE      | Development/debugging                   |
| MediaPipe          | Optional gesture recognition            |
| PlantUML / draw.io | UML/architecture diagrams when required |
| pytest / unittest  | Testing                                 |